

Computer Vision – Lecture 4

Image Processing III

23.04.2024

Flipped classroom – Q&A session

Bastian Leibe

Visual Computing Institute

RWTH Aachen University

<http://www.vision.rwth-aachen.de/>

leibe@vision.rwth-aachen.de

“Flipped Classroom” Model

Lecture 4: Image Processing III (26.04.2021)

 cv21-part04-image-processing3.pdf
2.1MB Hochgeladen 25.04.2021 15:04

 cv21-part04-image-processing3-6on1.pdf
1.3MB Hochgeladen 25.04.2021 15:04

 Pre-recorded videos

"Flipped classroom" model: Please watch the following pre-recorded videos before the lecture on Monday 26.04. The lecture slot will then contain a Q&A on the topics covered in the videos.

Pre-recorded videos

Part 3.3: Image Gradients



Computer Vision – Lecture 3

Image Processing II

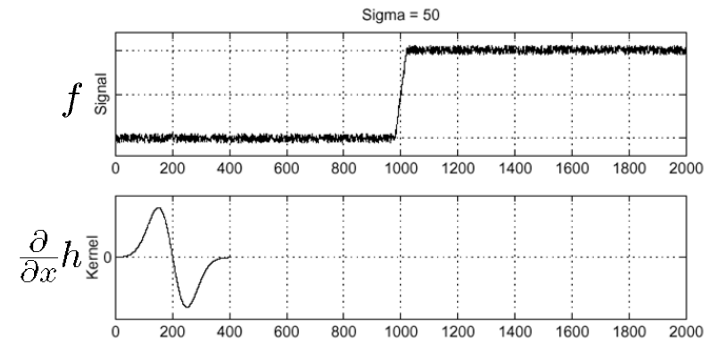
3.3 Image Gradients

Bastian Leibe
Visual Computing Institute
RWTH Aachen University
<http://www.vision.rwth-aachen.de/>

- Pre-recorded videos made available on moodle
 - Part 3.3: [Image Gradients](#)
 - Part 4.1: [Edge Detection](#)
- *Quick poll: Who watched the videos?*

Course Outline

- Image Processing Basics
 - Image Formation
 - Linear Filters
 - Gradient Filters and Edge Detection
 - Structure Extraction
- Segmentation
- Object Recognition and Categorization
- Deep Learning Approaches for Vision
- 3D Reconstruction



Derivative Filters

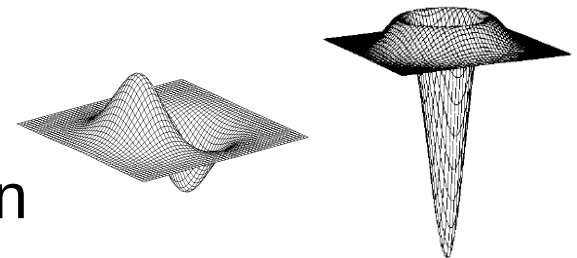
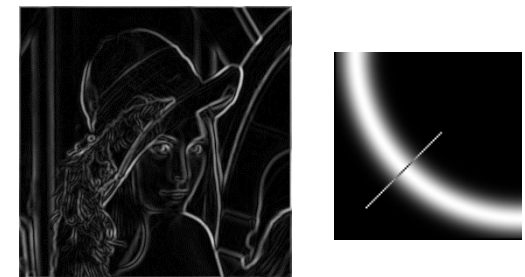


Image Derivative Operators



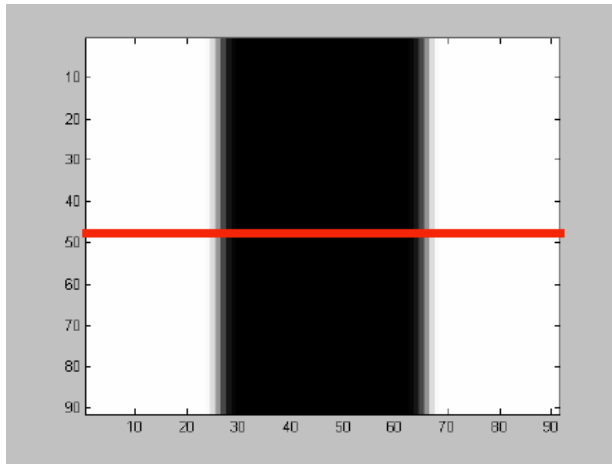
Canny Edge Detector

Topics of This Lecture

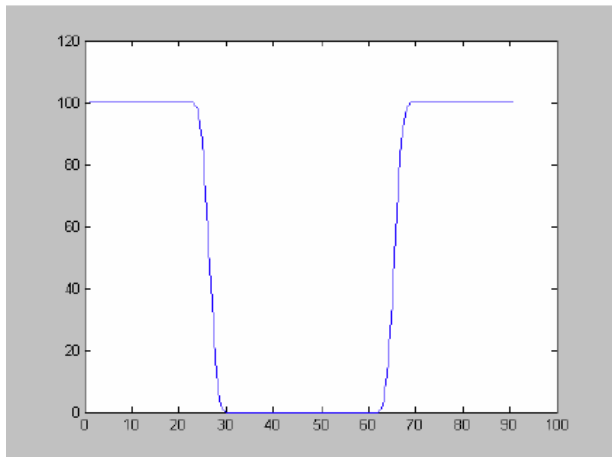
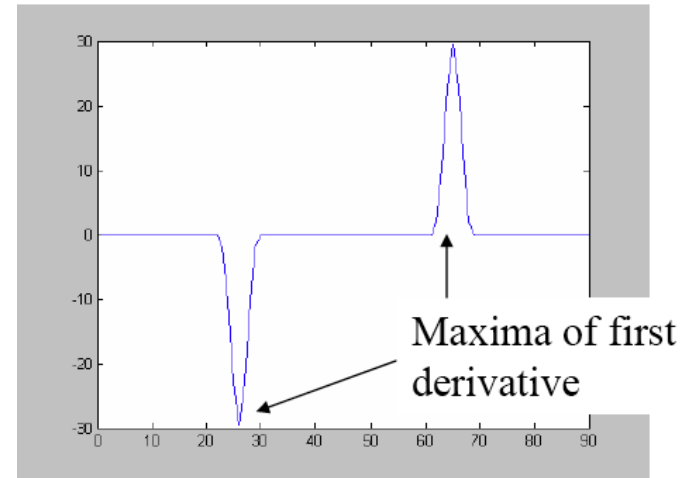
- Edge detection
 - Recap: Image gradients
 - Canny edge detector
- Structure extraction
 - Line detection
- Hough Transform
 - Derivation
 - Extensions
 - Generalized Hough transform



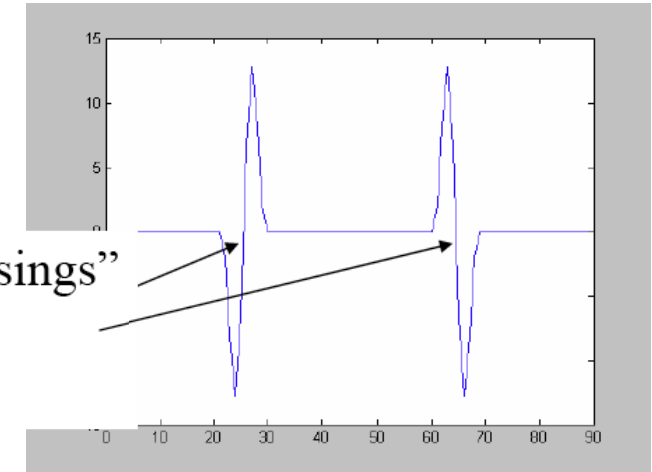
Recap: Derivatives and Edges...



1st derivative



2nd derivative



Recap: Finite Difference Filters

- For the 2D function $f(x, y)$, the partial derivative is
 - By definition:
$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$
- Approximation for discrete data using **finite differences**

- Simplest case
$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$
- More elaborate finite-difference approximations are available

- Implementation as a convolutional filter

- Filter mask:

1	-1
---	----

Sobel

$$M_x =$$

-1	0	1
-2	0	2
-1	0	1

$$M_y =$$

1	2	1
0	0	0
-1	-2	-1

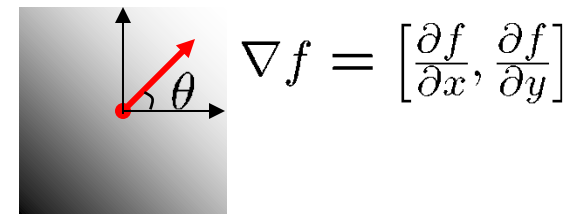
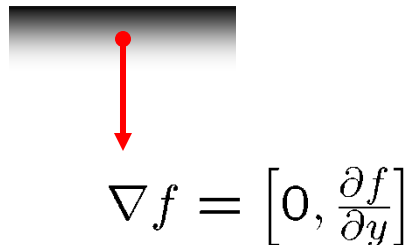
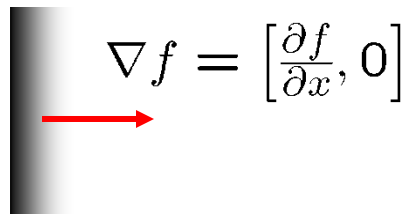
- In practice, use filters with an odd number of entries, e.g.,

Recap: Image Gradient

- The gradient of an image:

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

- The gradient points in the direction of most rapid intensity change



- The gradient direction (orientation of edge normal) is given by:

$$\theta = \tan^{-1} \left(\frac{\partial f / \partial y}{\partial f / \partial x} \right)$$

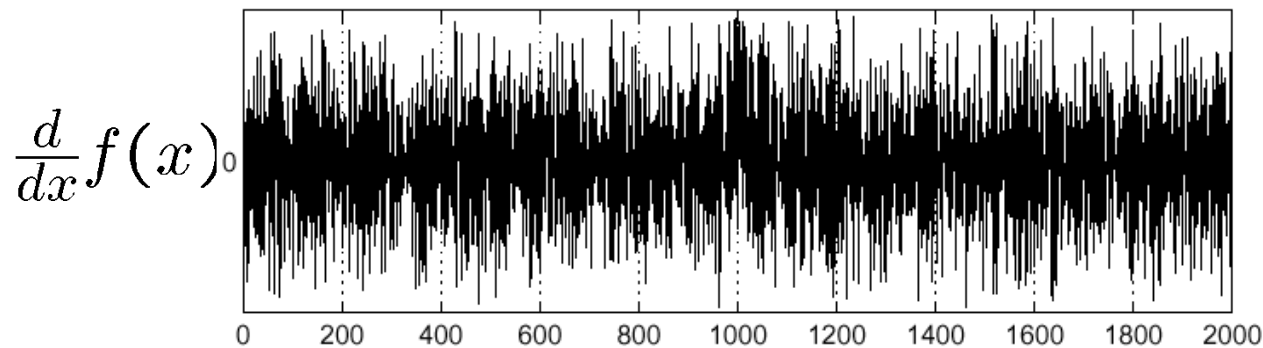
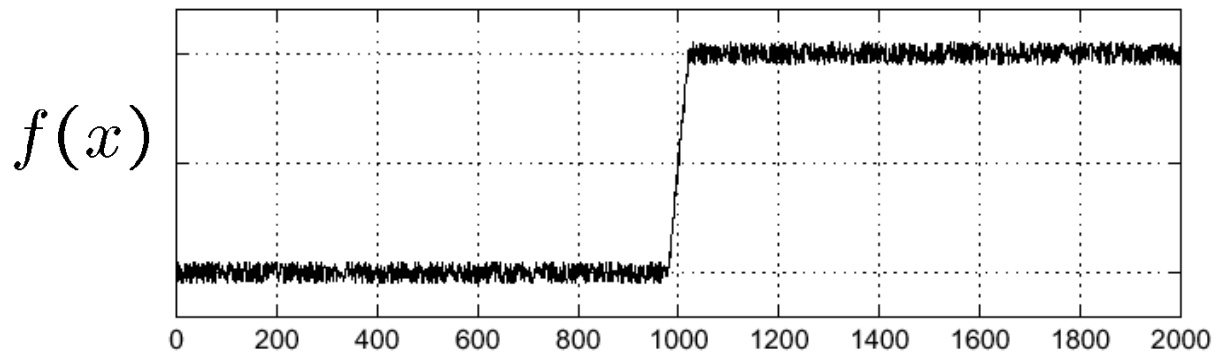
- The edge strength is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$



Recap: Effect of Noise

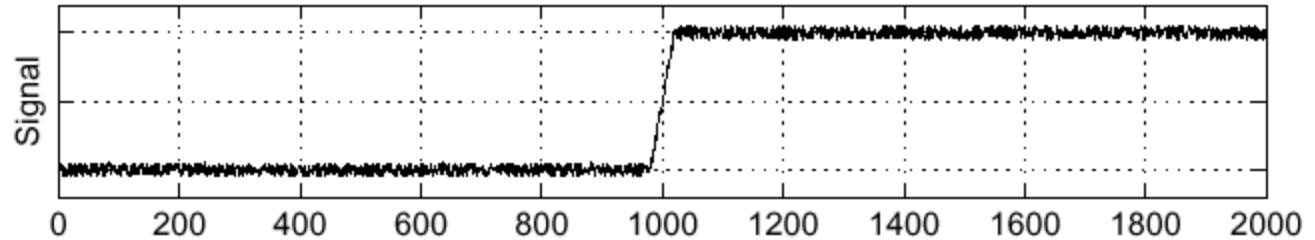
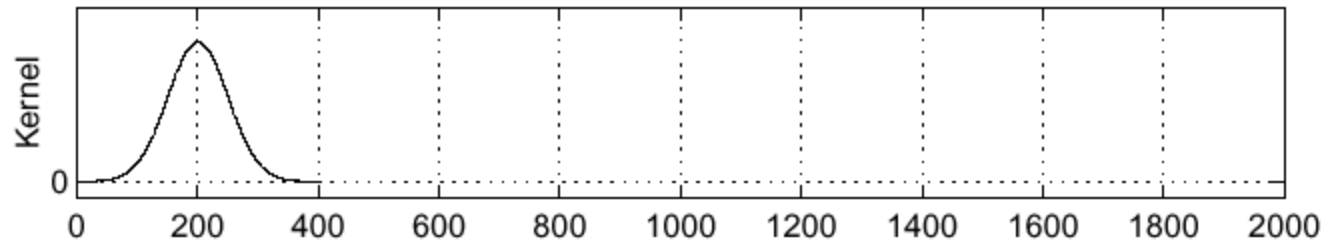
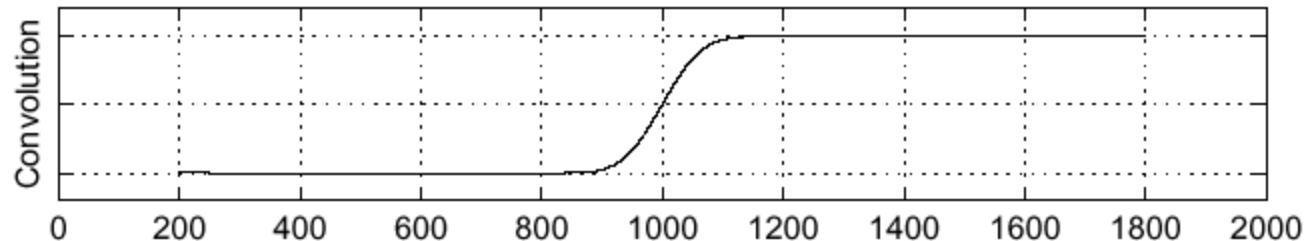
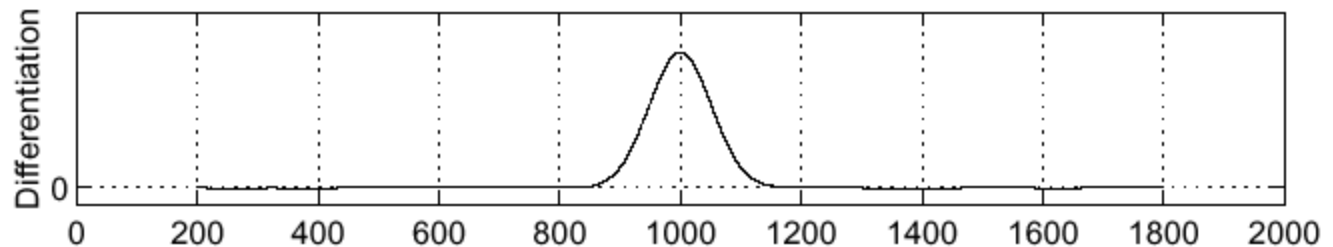
- Consider a single row or column of the image
 - Plotting intensity as a function of position gives a signal



Where is the edge?

Recap: Solution: Smooth First

Sigma = 50

 f  h  $h \star f$  $\frac{\partial}{\partial x}(h \star f)$ 

Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

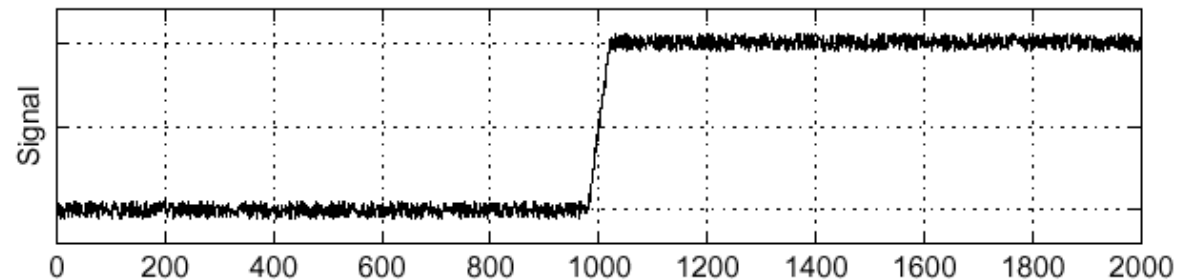
Recap: Derivative of Gaussian Filters

- Differentiation property of convolution.

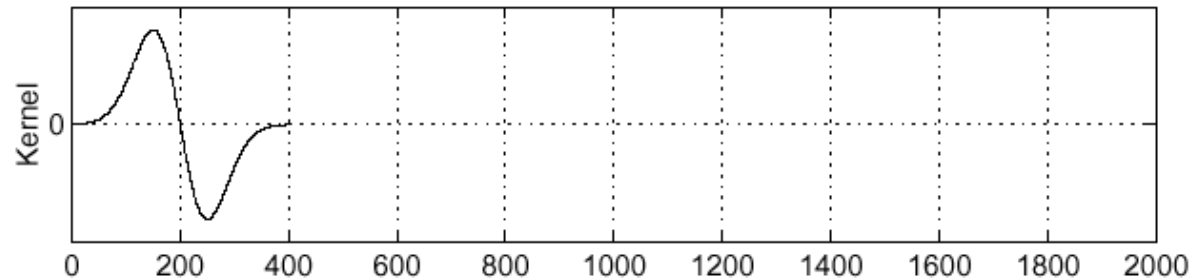
$$\frac{\partial}{\partial x}(h \star f) = \left(\frac{\partial}{\partial x}h\right) \star f$$

Sigma = 50

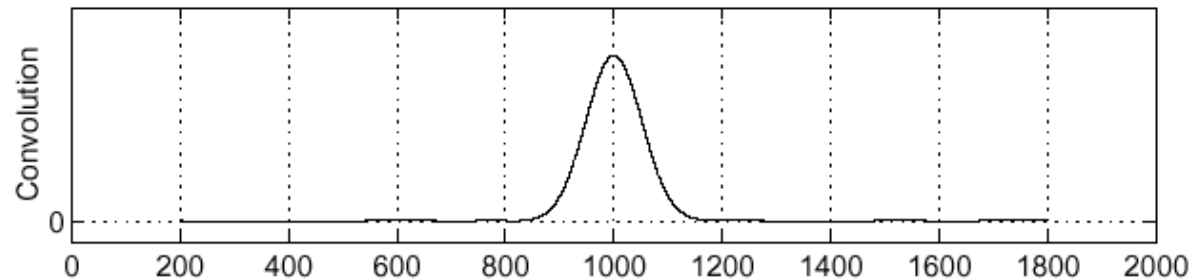
f



$\frac{\partial}{\partial x}h$



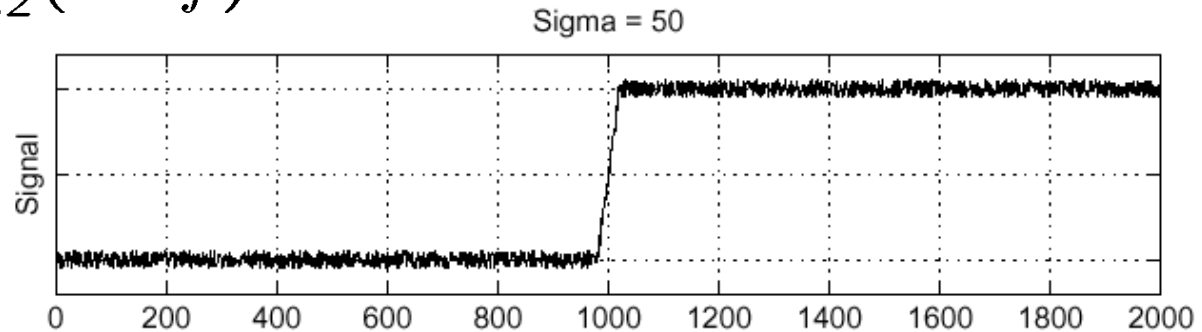
$\left(\frac{\partial}{\partial x}h\right) \star f$



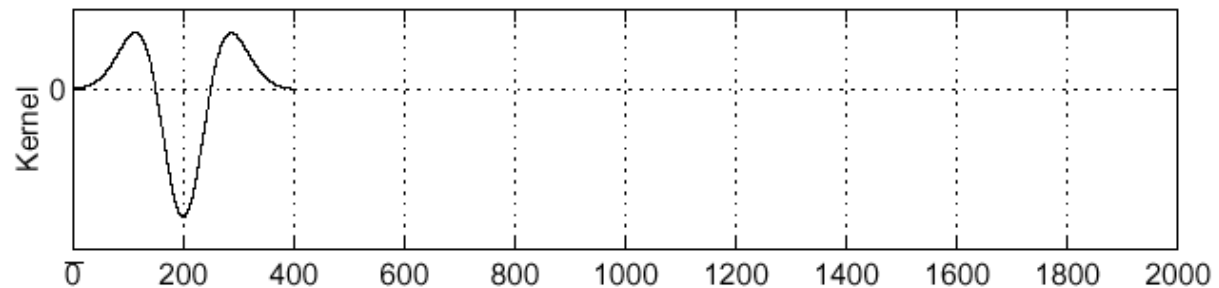
Recap: Laplacian of Gaussian (LoG)

- Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

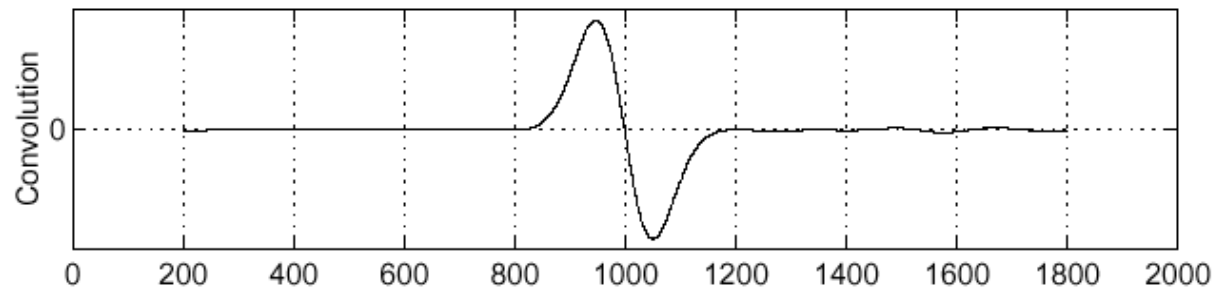
f



$\frac{\partial^2}{\partial x^2}h$



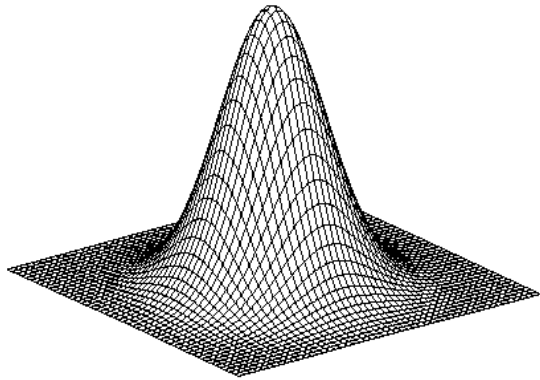
$(\frac{\partial^2}{\partial x^2}h) \star f$



Where is the edge?

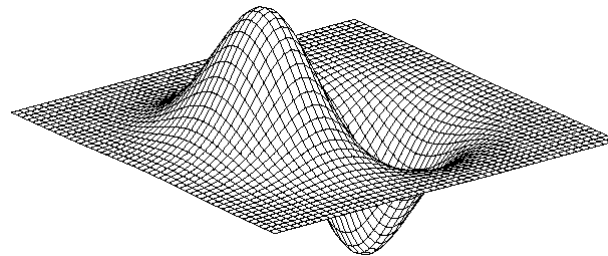
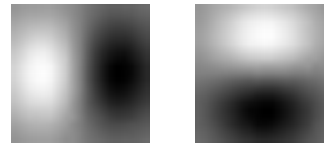
Zero-crossings of bottom graph

Recap: 2D Edge Detection Filters



Gaussian

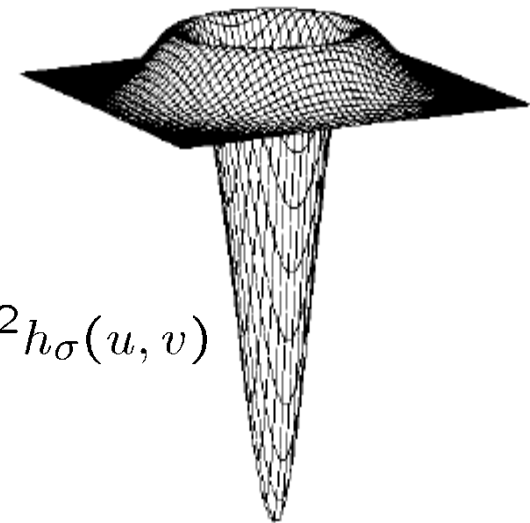
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



Derivative of Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian



$$\nabla^2 h_{\sigma}(u, v)$$

- ∇^2 is the Laplacian operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Topics of This Lecture

- Edge detection
 - Recap: Image gradients
 - Canny edge detector
- Structure extraction
 - Line detection
- Hough Transform
 - Derivation
 - Extensions
 - Generalized Hough transform



Recap: Edge Detection

- Goal: map image from 2D array of pixels to a set of curves or line segments or contours.
- Why?

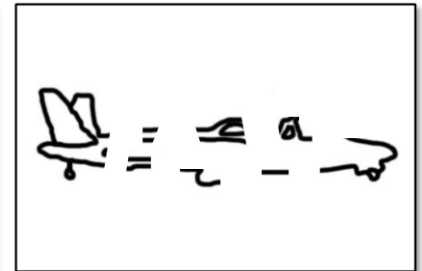
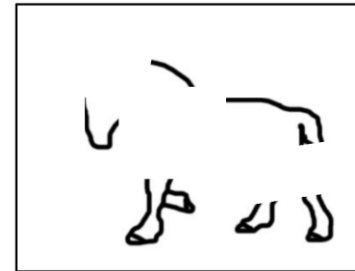
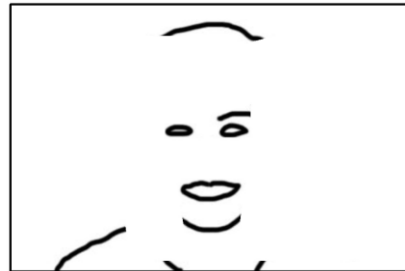


Figure from J. Shotton et al., PAMI 2007

- Main idea: look for strong gradients, post-process

Recap: Designing an Edge Detector

- Criteria for an “optimal” edge detector:
 - **Good detection**: the optimal detector should minimize the probability of false positives (detecting spurious edges caused by noise), as well as that of false negatives (missing real edges).
 - **Good localization**: the edges detected should be as close as possible to the true edges.
 - **Single response**: the detector should return one point only for each true edge point; that is, minimize the number of local maxima around the true edge.



True
edge



Poor robustness
to noise



Poor
localization



Too many
responses

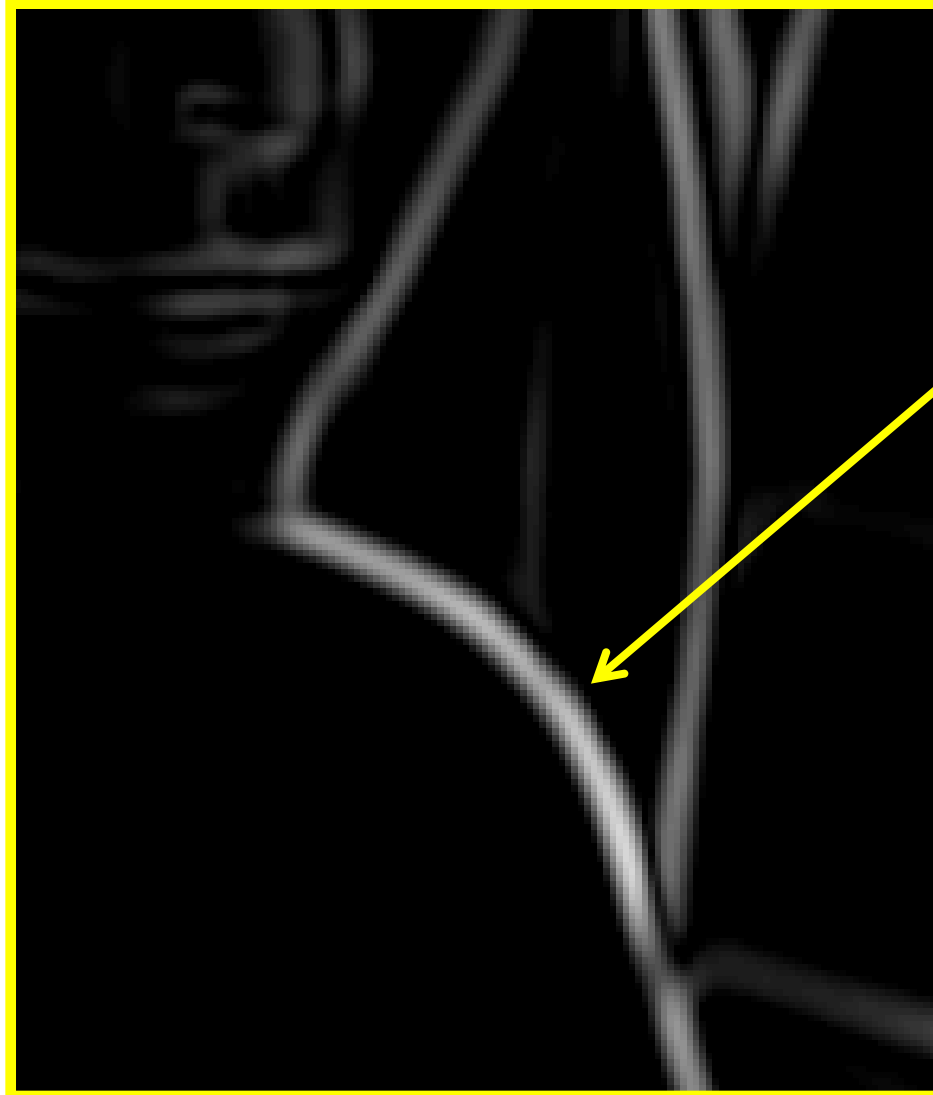
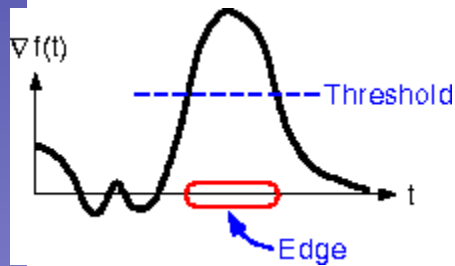
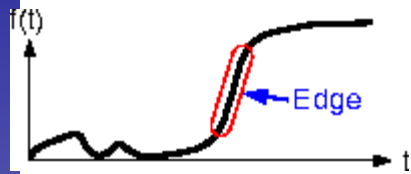
Recap: Canny Edge Detector

1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum suppression:
 - Thin multi-pixel wide “ridges” down to single pixel width
4. Linking and thresholding (hysteresis):
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them



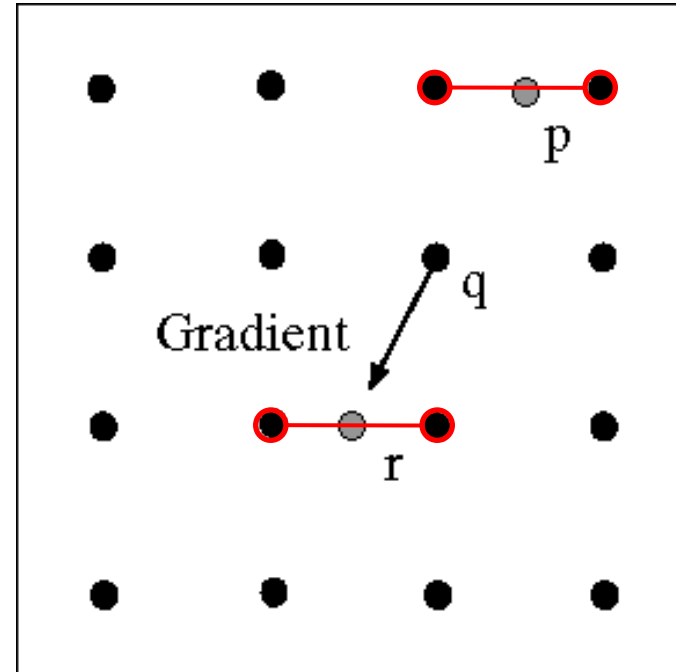
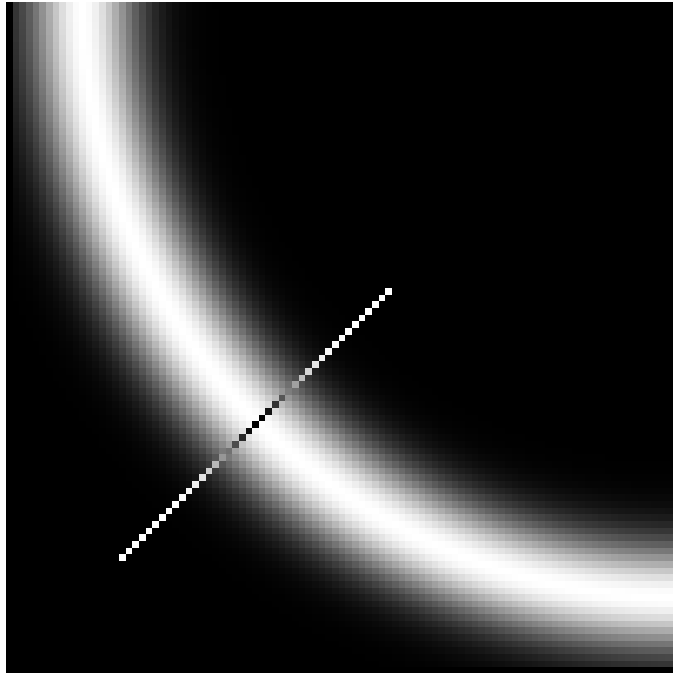
adapted from D. Lowe, L. Fei-Fei

Recap: Multi-Pixel Gradient Responses



How to turn these thick regions of the gradient into single-pixel curves?

Recap: Non-Maximum Suppression

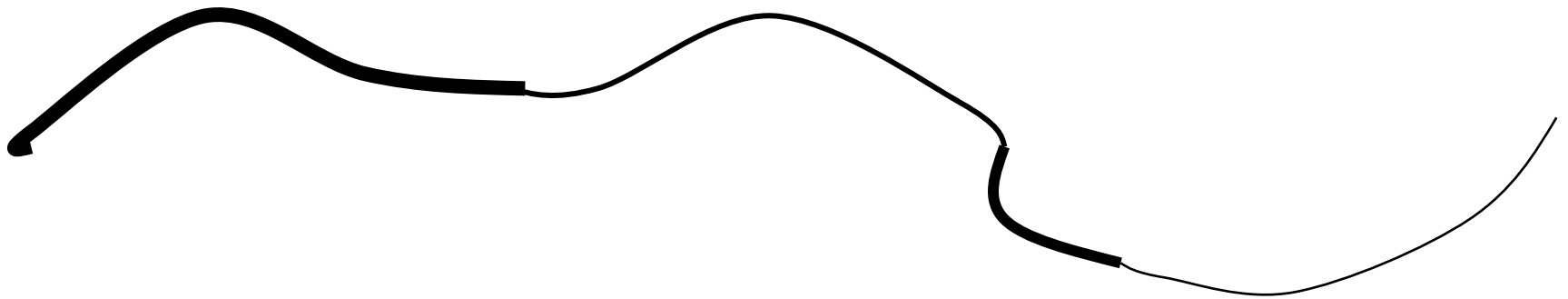


- Check if pixel is local maximum along gradient direction, select single max across width of the edge
 - I.e., keep q iff $\text{Mag}(q) > \text{Mag}(p)$ and $\text{Mag}(q) > \text{Mag}(r)$.
 - Requires checking interpolated pixels p and r $\Rightarrow$ Linear interpolation based on gradient direction

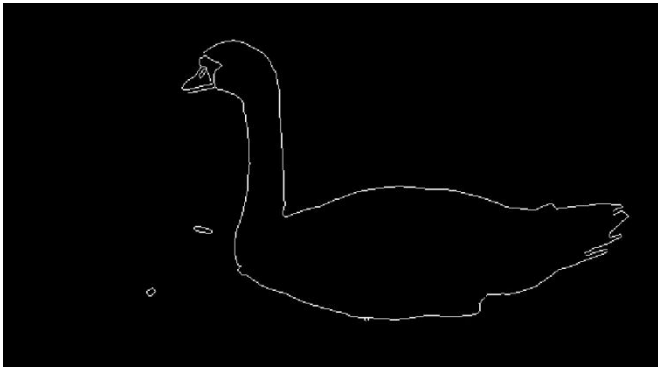
Recap: Hysteresis Thresholding

- Hysteresis: A lag or momentum factor
- Idea: Maintain two thresholds k_{high} and k_{low}
 - Use k_{high} to find strong edges to start edge chain
 - Use k_{low} to find weak edges which continue edge chain
- Typical ratio of thresholds is roughly

$$k_{high} / k_{low} = 2$$



Recap: Edges vs. Boundaries



Edges useful signal to indicate occluding boundaries, shape.

Here the raw edge output is not so bad...



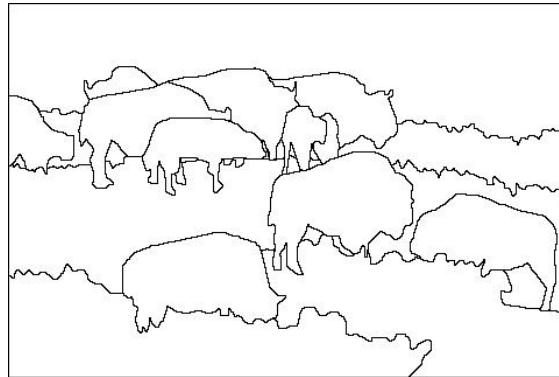
...but quite often boundaries of interest are fragmented, and we have extra “clutter” edge points.

Edge Detection is Just the Beginning...

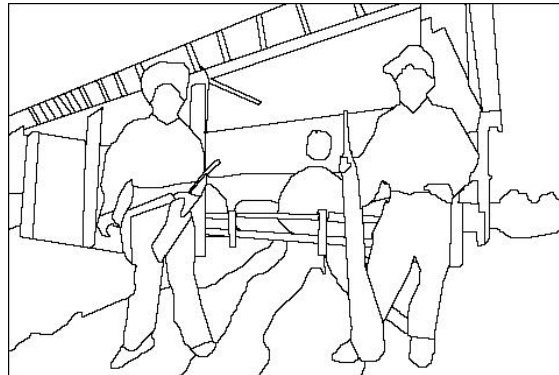
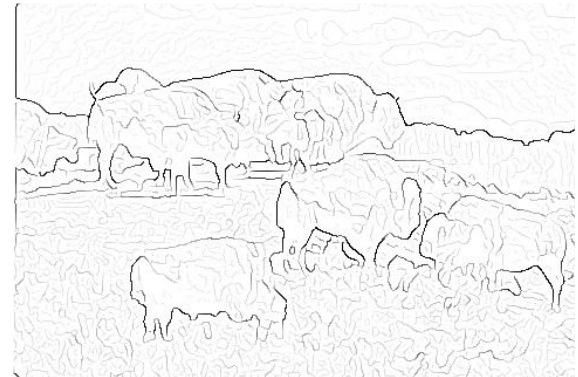
Image



Human segmentation



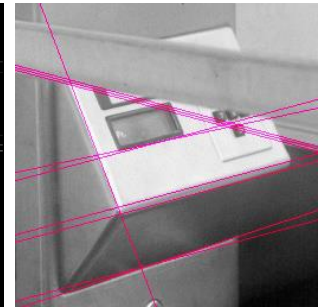
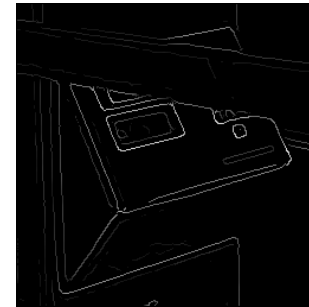
Gradient magnitude



- Berkeley segmentation database:
<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>

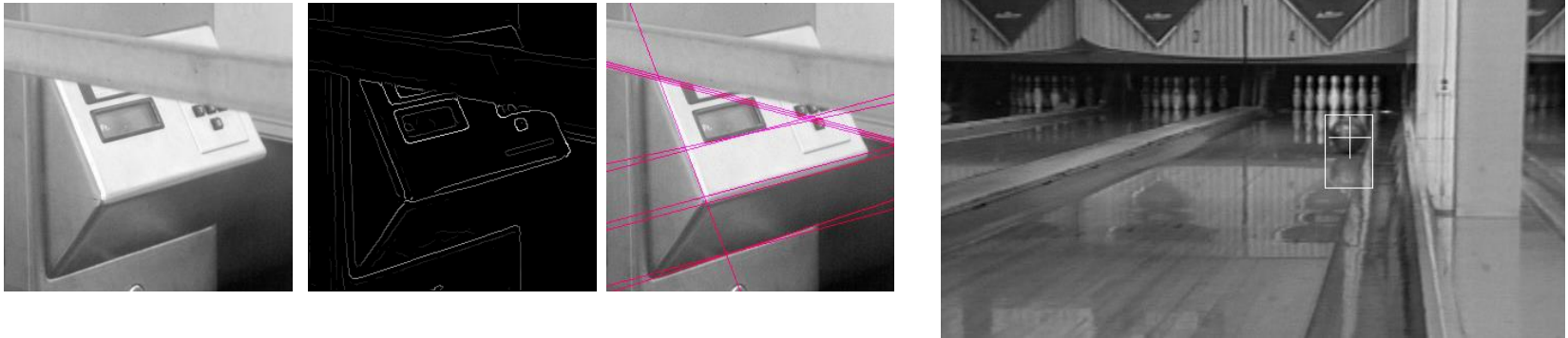
Topics of This Lecture

- Edge detection
 - Recap: Image gradients
 - Canny edge detector
- Structure extraction
 - Line detection
- Hough Transform
 - Derivation
 - Extensions
 - Generalized Hough transform



Fitting

- Want to associate a model with observed features

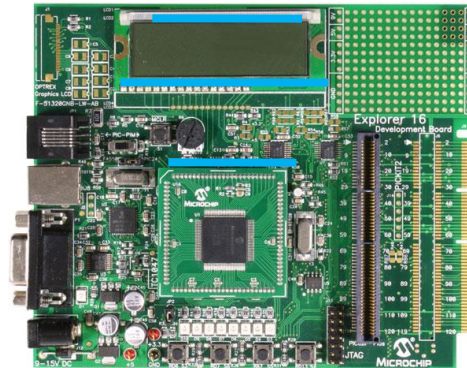


[Figure from Marszalek & Schmid, 2007]

For example, the model could be a line, a circle, or an arbitrary shape.

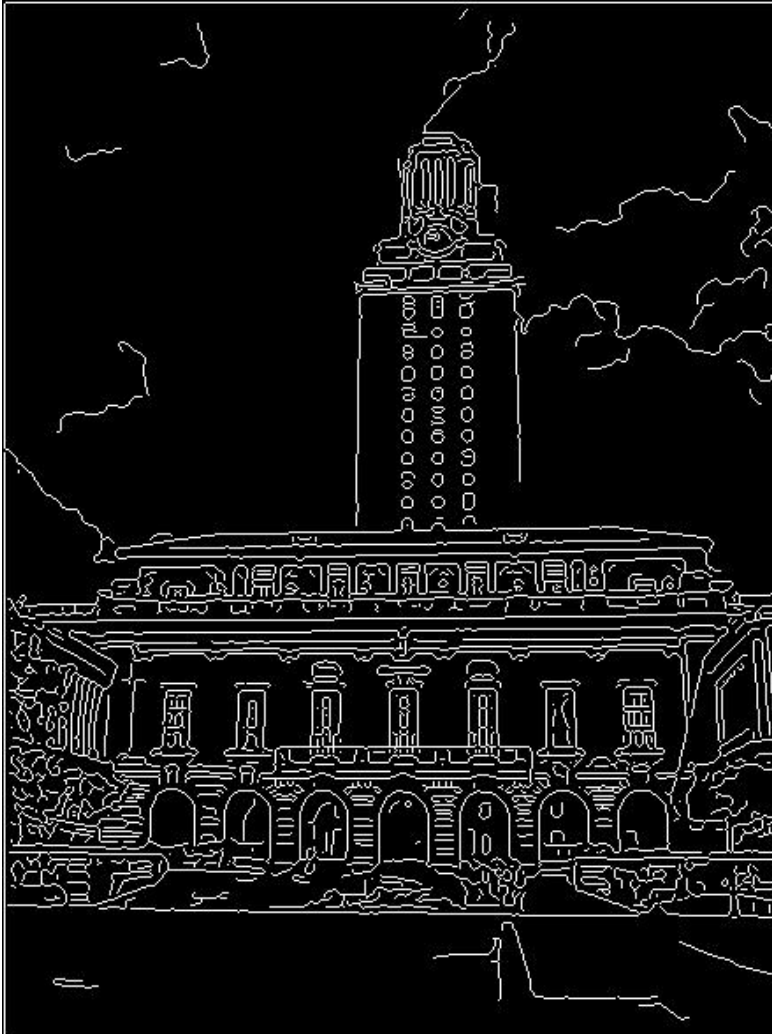
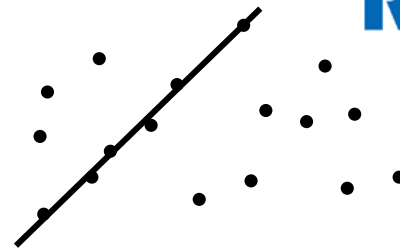
Example: Line Fitting

- Why fit lines?
 - Many objects are characterized by presence of straight lines



- Wait, why aren't we done just by running edge detection?

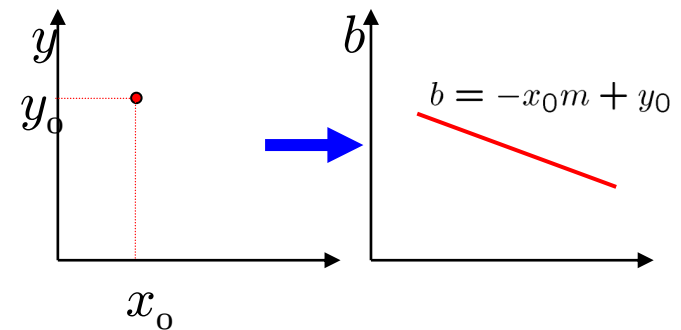
Difficulty of Line Fitting



- Extra edge points (clutter), multiple models:
 - Which points go with which line, if any?
- Only some parts of each line detected, and some parts are missing:
 - How to find a line that bridges missing evidence?
- Noise in measured edge points, orientations:
 - How to detect true underlying parameters?

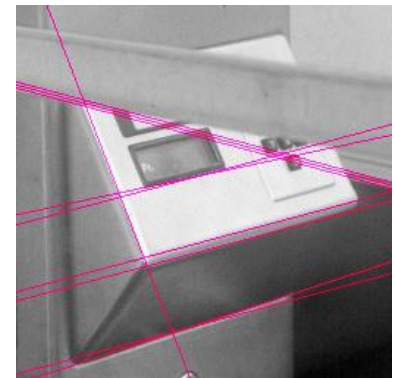
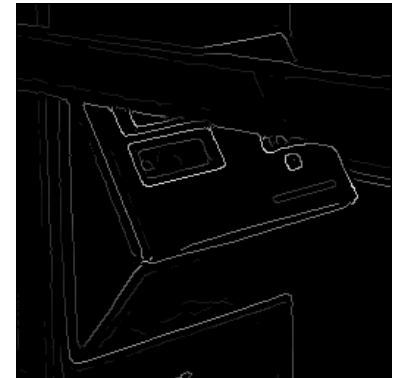
Topics of This Lecture

- Edge detection
 - Recap: Image gradients
 - Canny edge detector
- Structure extraction
 - Line detection
- Hough Transform
 - Derivation
 - Extensions
 - Generalized Hough transform

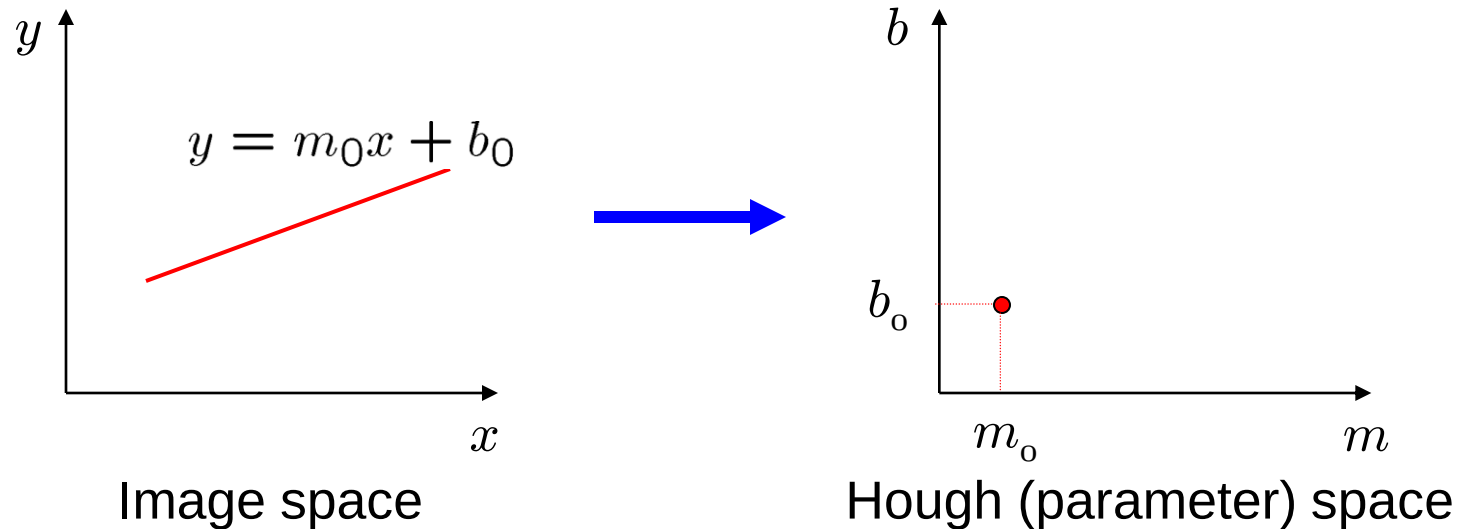


Fitting Lines

- Three main questions
 - Given points that belong to a line, what is the line?
 - How many lines are there?
 - Which points belong to which lines?
- The *Hough Transform* is a voting technique that can be used to answer all of these
- Main idea:
 1. Vote for all possible lines on which each edge point could lie.
 2. Look for line candidates that get many votes.
 3. Noise features will cast votes too, *but* their votes should be inconsistent

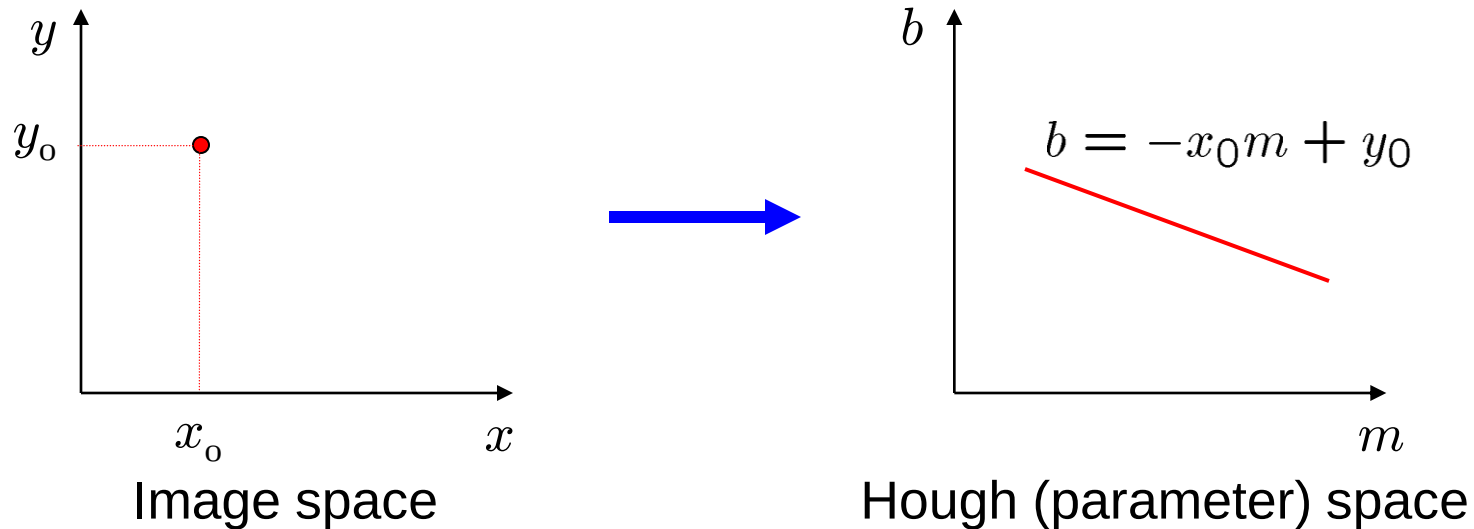


Finding Lines in an Image: Hough Space



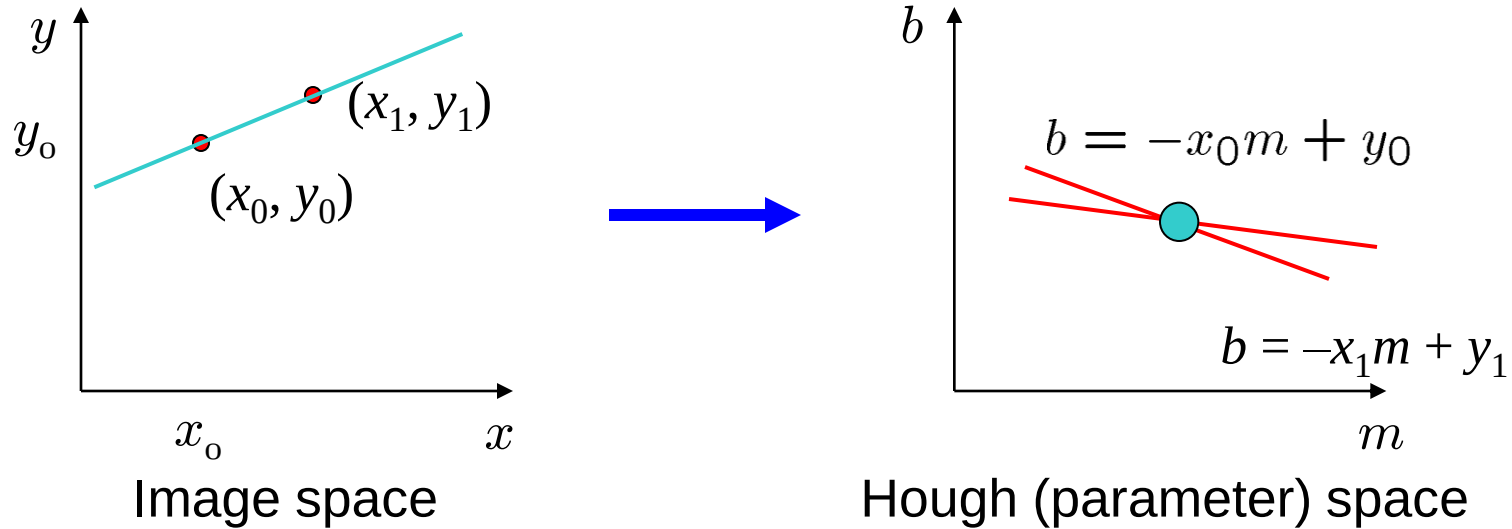
- Connection between image (x, y) and Hough (m, b) spaces
 - A line in the image corresponds to a point in Hough space.
 - To go from image space to Hough space:
 - Given a set of points (x, y) , find all (m, b) such that $y = mx + b$

Finding Lines in an Image: Hough Space



- Connection between image (x, y) and Hough (m, b) spaces
 - A line in the image corresponds to a point in Hough space.
 - To go from image space to Hough space:
 - Given a set of points (x, y) , find all (m, b) such that $y = mx + b$
 - What does a point (x_0, y_0) in the image space map to?
 - Answer: the solutions of $b = -x_0m + y_0$
 - This is a line in Hough space.

Finding Lines in an Image: Hough Space



- What are the line parameters for the line that contains both (x_0, y_0) and (x_1, y_1) ?
 - It is the intersection of the lines
$$b = -x_0 m + y_0 \text{ and } b = -x_1 m + y_1$$

Finding Lines in an Image: Hough Space

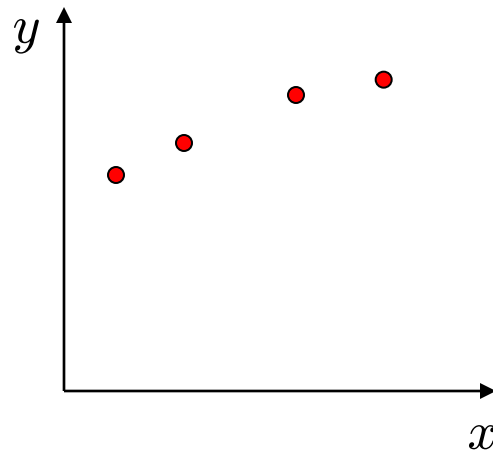
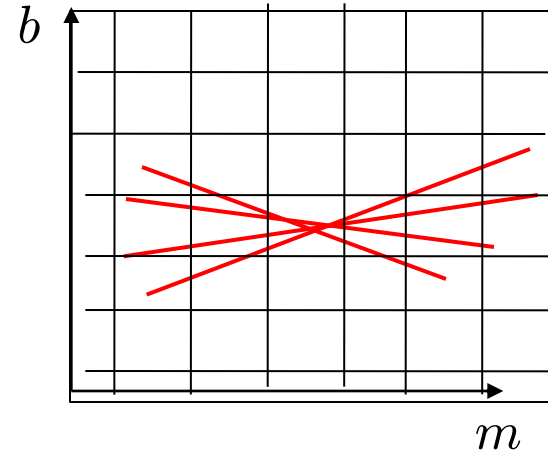


Image space

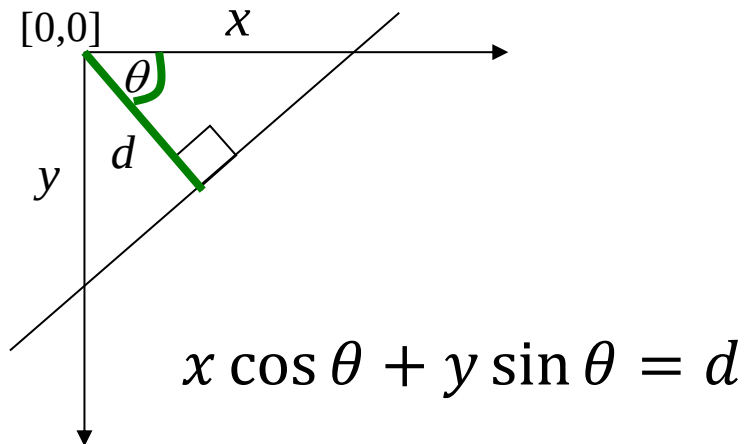


Hough (parameter) space

- How can we use this to find the most likely parameters (m, b) for the most prominent line in the image space?
 - Let each edge point in image space *vote* for a set of possible parameters in Hough space.
 - Accumulate votes in discrete set of bins; parameters with the most votes indicate line in image space.

Polar Representation for Lines

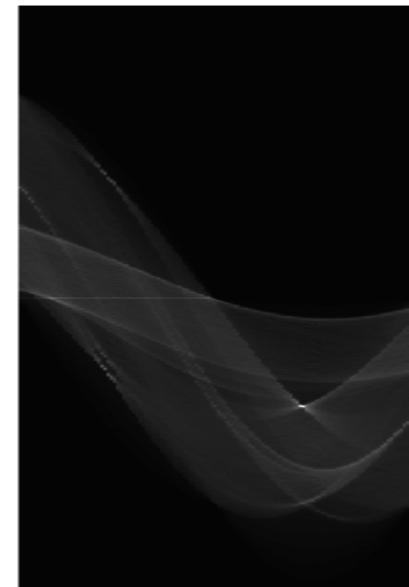
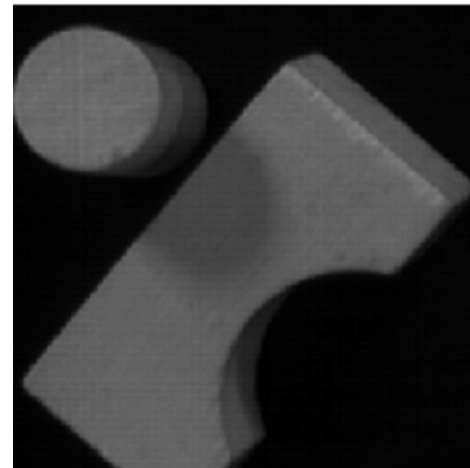
- Usual (m, b) parameter space problematic:
 - Can take on infinite values, undefined for vertical lines.



d : perpendicular distance from line to origin

θ : angle the perpendicular makes with the x-axis

- Point in image space
⇒ Sinusoid segment in Hough space



Hough Transform Algorithm

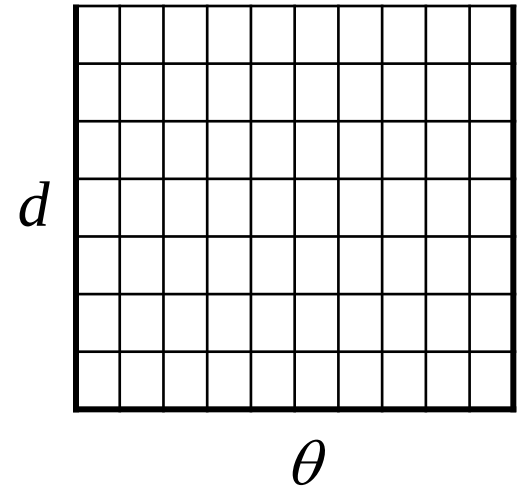
Using the polar parameterization:

$$x \cos \theta + y \sin \theta = d$$

Basic Hough transform algorithm

1. Initialize $H[d, \theta] = 0$.
2. For each edge point (x, y) in the image
for $\theta = 0$ to 180 // some quantization
 $d = x \cos \theta + y \sin \theta$
 $H[d, \theta] += 1$
3. Find the value(s) of $(\hat{d}, \hat{\theta})$ where $H[d, \theta]$ is maximal.
4. The detected line in the image is given by $\hat{d} = x \cos \hat{\theta} + y \sin \hat{\theta}$

H : accumulator array (votes)



- Time complexity?

Example: HT for Straight Lines

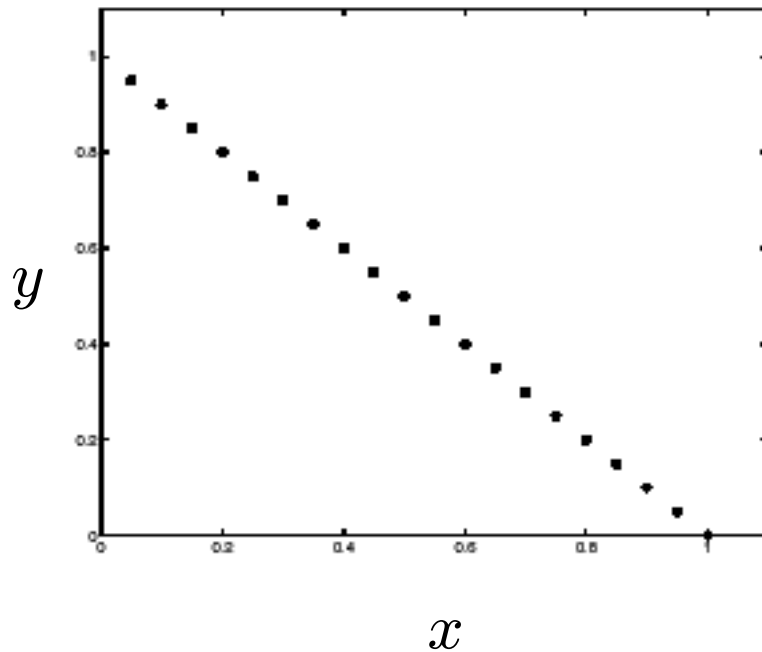
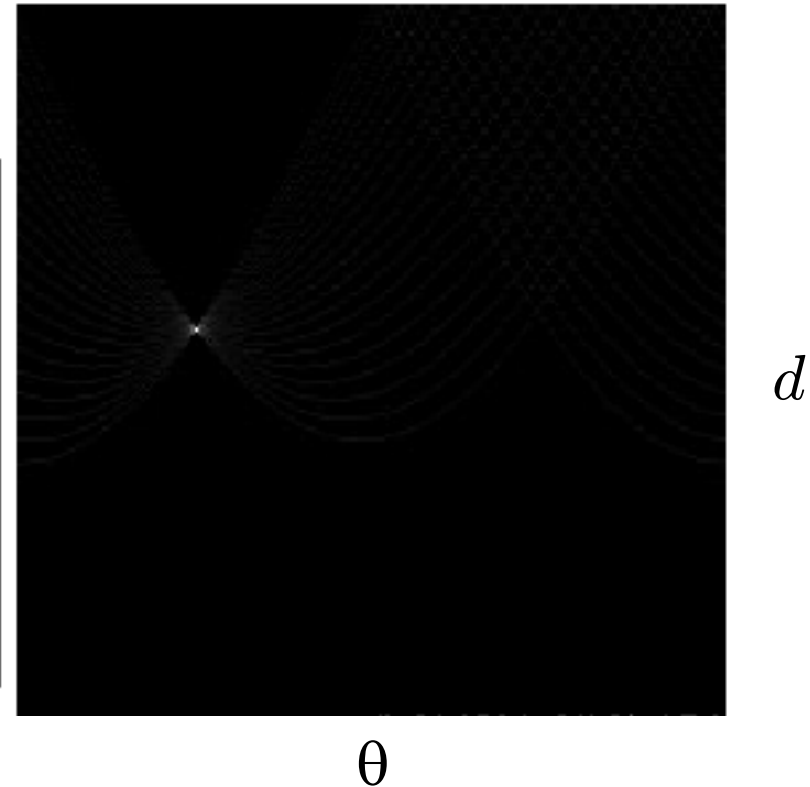


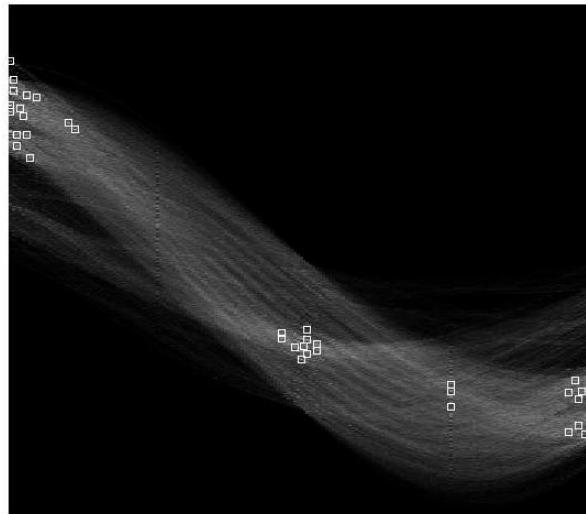
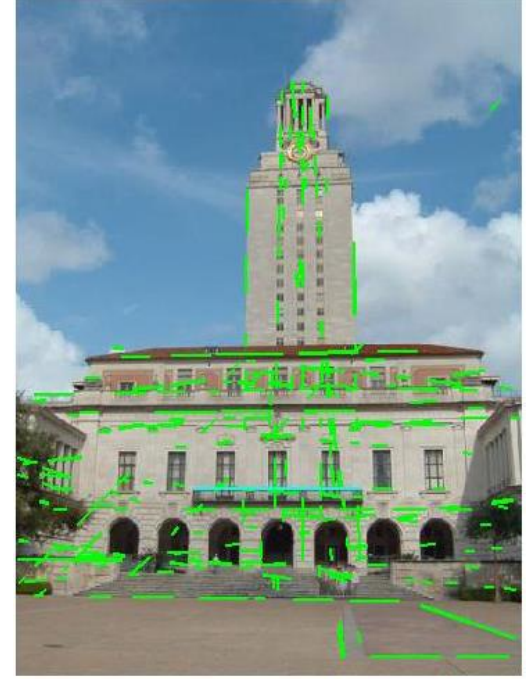
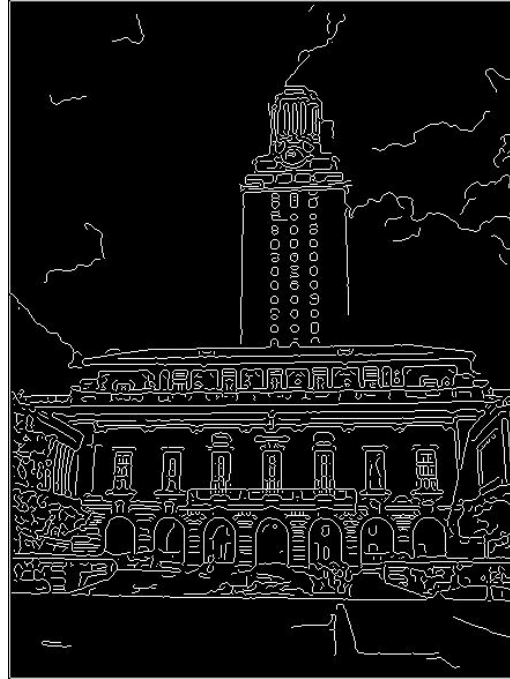
Image space
edge coordinates

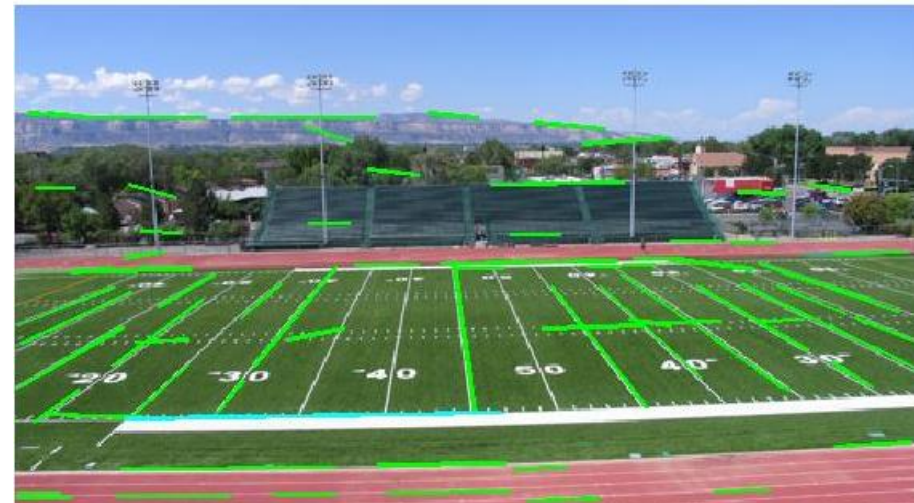
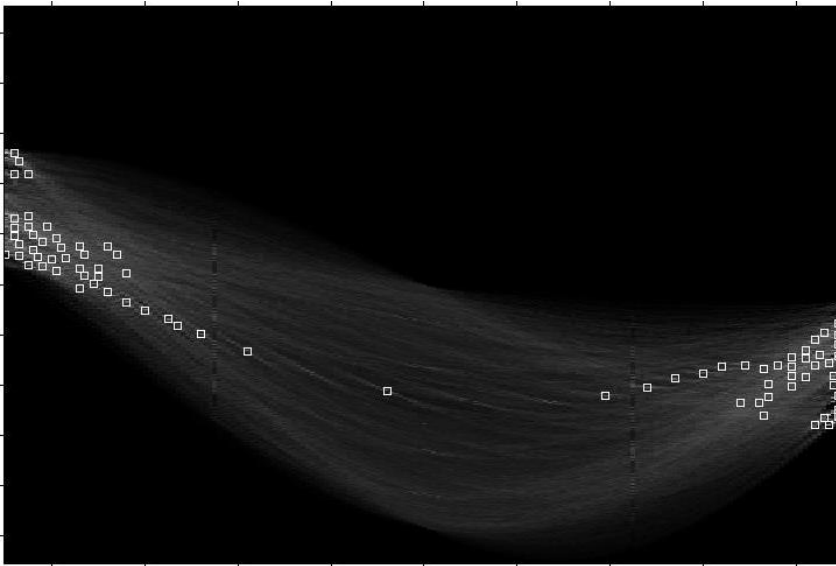


Votes

Bright value = high vote count
Black = no votes

Real-World Examples





Showing longest segments found

Impact of Noise on Hough Transform

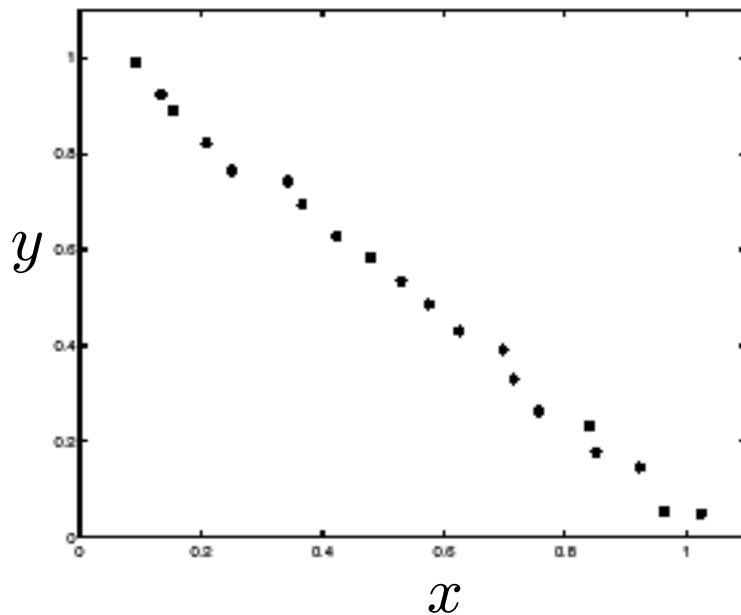
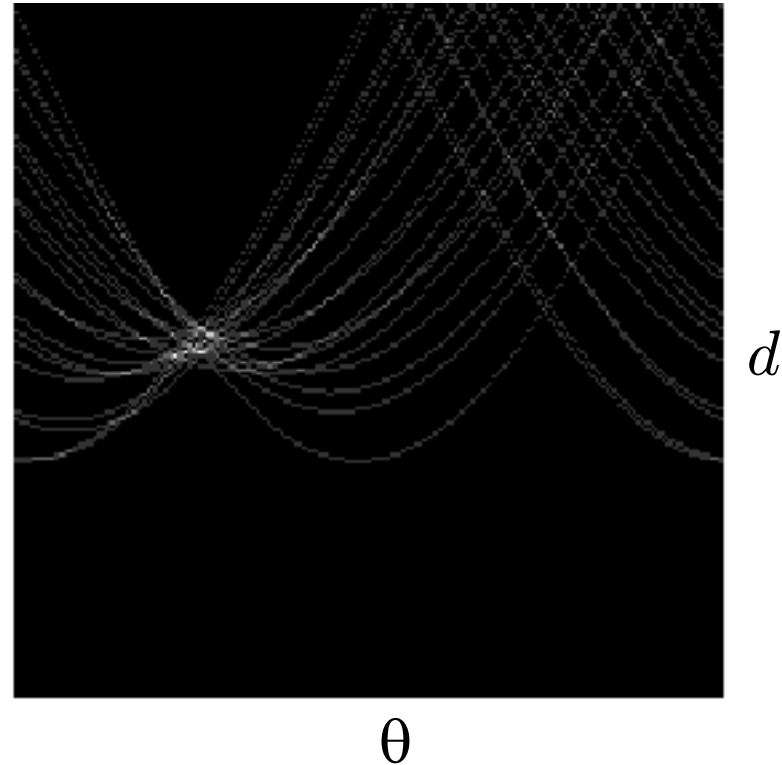


Image space
edge coordinates



Votes

What difficulty does this present for an implementation?

Impact of Noise on Hough Transform

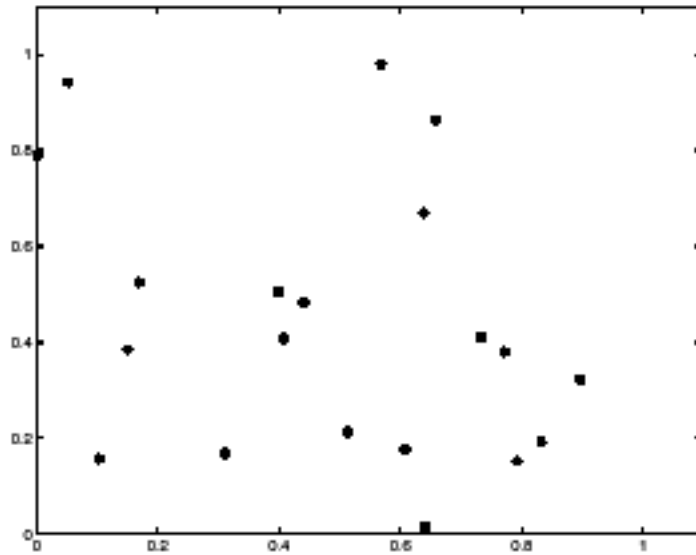
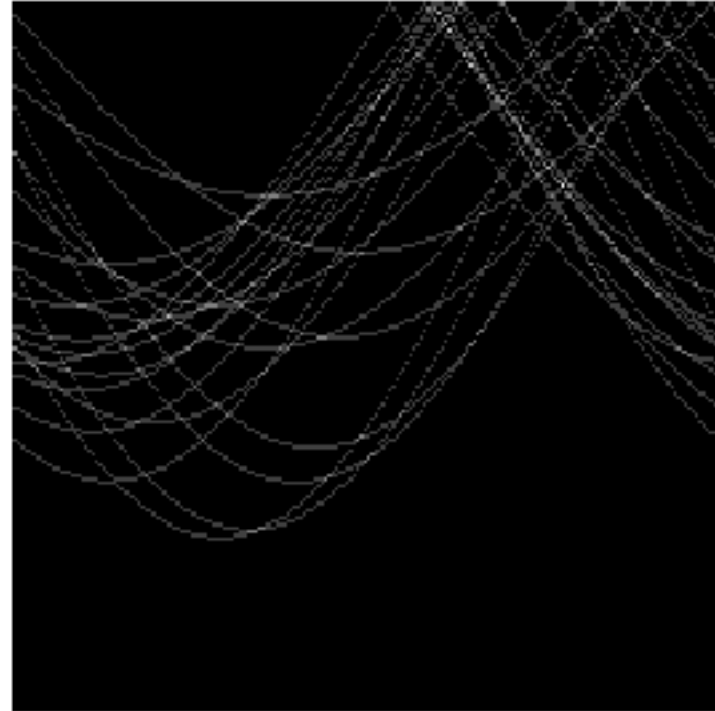


Image space
edge coordinates



Votes

Here, everything appears to be “noise”, or random edge points, but we still see peaks in the vote space.

Extensions

Extension 1: Use the image gradient

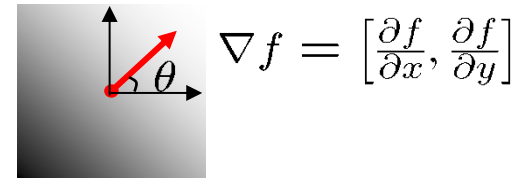
1. same
2. for each edge point $I[x, y]$ in the image
 $\theta = \text{gradient at } (x, y)$

$$d = x \cos \theta + y \sin \theta$$

$$H[d, \theta] += 1$$

3. same
4. same

(Reduces degrees of freedom)



$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

Extensions

Extension 1: Use the image gradient

1. same
2. for each edge point $I[x, y]$ in the image
compute unique (d, θ) based on image gradient at (x, y) .
 $H[d, \theta] += 1$
3. same
4. same

(Reduces degrees of freedom)

Extension 2

- Weight votes by the gradient magnitude (more weight for stronger edges)

Extension 3

- Change the sampling of (d, θ) to give more/less resolution

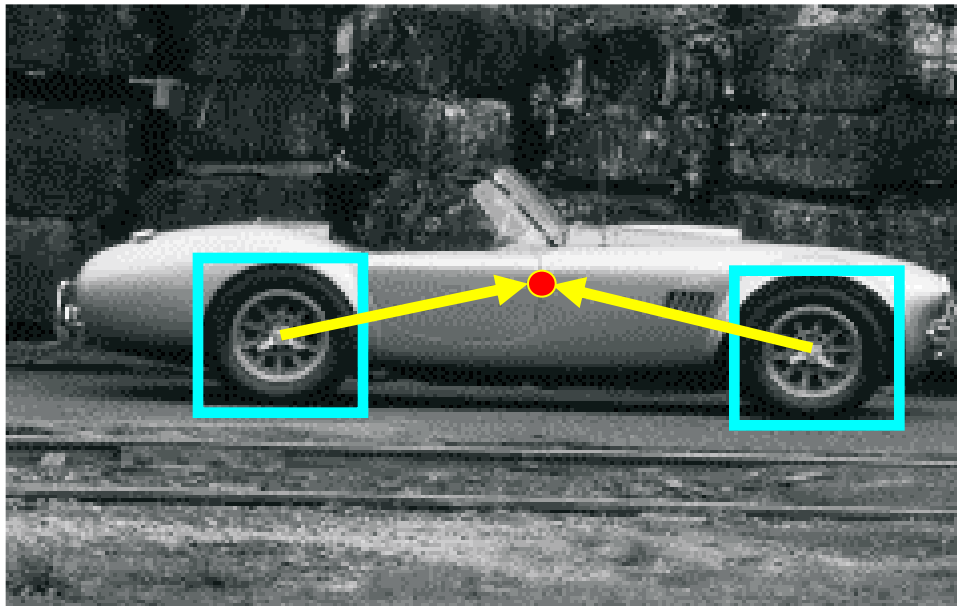
Extension 4

- The same procedure can be used with circles, ellipses or any other parametric shapes (vote for parameter values consistent with observations)

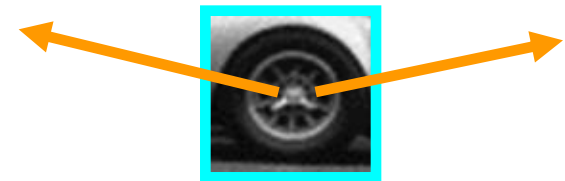
Application in Recognition

- Generalized Hough Transform

- Parametrize objects of a visual class by their center as $(x,y,scale)$
- Observations: detected high-level features (“visual codewords”).



Training image



Visual codeword with displacement vectors

B. Leibe, A. Leonardis, and B. Schiele, [Robust Object Detection with Interleaved Categorization and Segmentation](#), International Journal of Computer Vision, Vol. 77(1-3), 2008.

Application in Recognition

- Generalized Hough Transform

- Parametrize objects of a visual class by their center as $(x,y,scale)$
- Observations: detected high-level features (“visual codewords”).



Test image

- Possible mechanism for designing object detectors...

References and Further Reading

- Background information on image gradients is available in Chapter 3; background on edge detection can be found in Chapter 4 of the Szeliski book.

R. Szeliski
Computer Vision – Algorithms and Applications
Springer, 2010
([draft version available online](#))

